

</ Session 2 Decisions and Repetition

/>

} /> [

Information Systems
Development (SE)
Dr. Pejman Ebrahimi



{01}

Recap and warm-up

{02}

Conditions

{03}

Repetition

{04}

Lists

{05}

The guessing game

{06}

Your turn

</

/> **

Last week

Variables, text and numbers.
input() and int(). Reading an error
from the bottom up.

You built Mad Libs and the room
laughed.

} /> [

This week

Programs that make a decision, and
programs that repeat themselves.

Both of those together give you a
game.



```
items = 10
```

```
boxes = 3
```

```
print(items / boxes)      #
```

```
3.3333333333333335
```

```
print(items // boxes)    # 3   full boxes
```

```
print(items % boxes)     # 1   left over
```

Same ritual

Read the problem.

Say it back in one sentence.

Name the inputs and the outputs.

Only then type.

</>

Conditions

01

} /> [

</

/> **

A boolean

A value with only two possibilities: True or False.

Capital T, capital F. No quotes – they are not text.

} /> [

Where they come from

Any comparison produces one.

`stock > 0` is a question, and Python answers it with True or False.

</

==

Is it equal?
4 == 4 is True

!=

Is it different?
4 != 5 is True

<

Smaller than
3 < 5 is True

>

Bigger than
3 > 5 is False

<=

Smaller or equal
5 <= 5 is True

>=

Bigger or equal
6 >= 5 is True

</

/> **

= gives

stock = 4

An instruction. Put the value 4
into the name stock.
Nothing comes back.

} /> [

== asks

stock == 4

A question. Is what is in stock
equal to 4?
True or False comes back.



```
stock = 4
```

```
if stock == 0:  
    print("Sold out")  
elif stock < 5:  
    print("Only a few left")  
else:  
    print("In stock")
```

Watch these

The colon at the end of the line.

The four spaces of indent – Python reads them.

elif is only checked if the line above was False.

else catches everything left over.



```
total = 120  
member = True
```

```
if total >= 100 and member:  
    discount = 0.20  
elif total >= 100:  
    discount = 0.10  
else:  
    discount = 0.0
```

```
print(f"You pay {total * (1 - discount)} EUR")
```

Why this order

Python stops at the first branch that is True.

The most specific rule has to come first.

Swap the first two and members never get 20%.



and

Both sides must be True.

```
total >= 100 and  
member
```

A big order from a member. One of the two is not enough.

or

One side is enough.

```
member or first_order
```

Members get it. First-time buyers get it. Both get it.

not

Flips it round.

```
not member
```

True becomes False, False becomes True. Use it sparingly.



02

} /> [

</

```
guesses = 0
```

```
while guesses < 3:  
    print(f"Attempt {guesses + 1}")  
    guesses = guesses + 1  
  
print("Out of attempts")
```

How it runs

Check the condition.

If True, run the indented block.

Go back to the top and check again.

If False, skip to the line after.

</

/> **

This never stops

```
guesses = 0
```

```
while guesses < 3:  
    print(guesses)
```

Nothing changes guesses, so the condition stays True forever.

} /> [

The saving habit

Before you run a while loop, point at the line that will eventually make the condition False.

If you cannot find it, it is not there.

To stop a runaway cell in Colab: the square Stop button.



```
products = ["Laptop", "Mouse", "Cable"]
```

```
for item in products:  
    print(item)
```

```
for n in range(3):  
    print(n)           # 0, 1, 2
```

for vs while

for walks through a collection.

It stops on its own – no counter to forget.

range(3) gives 0, 1, 2. It stops before 3.

</

```
prices = [899.0, 24.5, 9.99]
```

```
total = 0
```

```
for p in prices:
```

```
    total = total + p
```

```
print(f"Basket total: {total} EUR")
```

The shape

Start empty, outside the loop.

Add one thing per turn, inside the loop.

Read the answer after the loop.

This shape returns in every session from now on.

</

/> **

break

Leave the loop right now.

You found what you were looking for, so there is no reason to keep going.

} /> [

continue

Skip the rest of this turn, go straight to the next one.

This item is not interesting – move on.

</

/> **

Use for when

You know the collection: every product, every row, every number from 1 to 10.

You are walking through something that already exists.

} /> [

Use while when

You do not know how many turns it will take.

Keep asking until the guess is right. Keep retrying until the connection works.

</>

Lists

03

} /> [



```
products = ["Laptop", "Mouse", "Cable"]
```

```
print(products[0])      # Laptop
print(products[-1])     # Cable
print(products[0:2])    # Laptop, Mouse
print(len(products))    # 3
```

Three habits

Square brackets make a list.

Square brackets also pick an item out.

Negative numbers count from the end.



	0	1	2	-1
products[...]	Laptop	Mouse	Cable	Cable
Reads as	first	second	third	last



```
basket = []
```

```
basket.append("Laptop")
```

```
basket.append("Mouse")
```

```
print(len(basket))      # 2
```

```
for item in basket:  
    print(f"- {item}")
```

Note the dot

`basket.append(x)` – the list does it to itself.

`len(basket)` – a function done to the list.

An empty list `[]` is a perfectly good start.



```
prices = [899.0, 24.5, 9.99, 149.0]
cheap  = []
```

```
for p in prices:
    if p < 100:
        cheap.append(p)
```

```
print(f"{len(cheap)} products under 100 EUR")
print(cheap)
```

Read it aloud

For every price in the list...
...if it is under a hundred...
...put it in the cheap list.
Two levels of indent. Count them.

</>

The build

04

} /> [

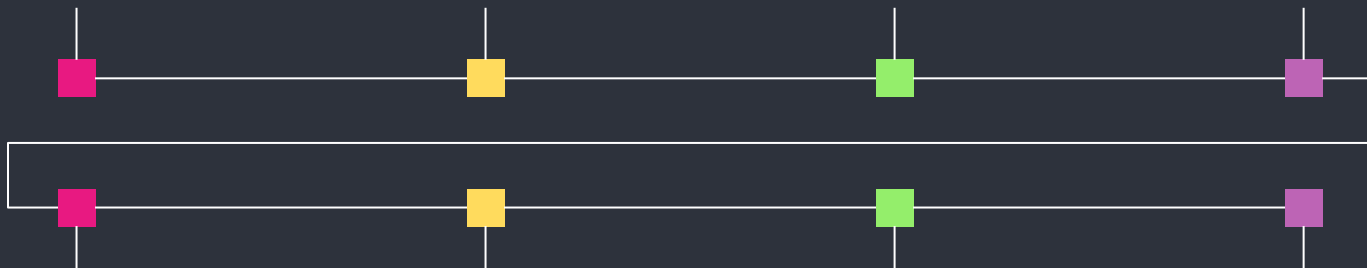


Pick a secret
number

Ask for one guess

Compare the two
with ==

Say correct or
wrong



Wrap it in a while
loop

Keep asking until
it is right

Add higher / lower
hints

Count the tries,
stop at five

</

```
secret = 7
```

```
guess = int(input("Your guess: "))
```

```
if guess == secret:  
    print("Correct!")
```

```
else:  
    print("Wrong.")
```

It works

It is boring, and that is fine.
One guess, one answer, no loop.
Run it twice before changing anything.

</

```
secret = 7
```

```
guess = 0
```

```
while guess != secret:  
    guess = int(input("Your guess: "))
```

```
print("Correct!")
```

Why guess = 0

The condition is checked before the first turn.

guess has to exist already.

0 is never the secret, so the loop always runs once.

</

```
import random
```

```
secret = random.randint(1, 20)  
tries = 0
```

```
while tries < 5:  
    guess = int(input("Guess: "))  
    tries = tries + 1  
  
    if guess == secret:  
        print(f"Got it in {tries}!")  
        break  
    elif guess < secret:  
        print("Higher")  
    else:  
        print("Lower")
```



Score it

Fewer tries, more points. Print the score at the end.

Difficulty levels

Ask easy, medium or hard first, then set the range and the number of tries.

Play again

Wrap the whole game in another while loop and ask 'again? y/n'.

Be rude to the player

Different messages for a wild guess and a close one. Warm, cold, boiling.

</>

Practice

05

} /> [



Easy

Even or odd. Count to ten. Biggest of three numbers.

Easy

Sum a list. Reverse a list. Count the vowels.

Medium

FizzBuzz. Multiplication table. Second largest.

Medium

Remove duplicates. Find the average. Grade a score.

Hard

Prime numbers. Palindromes. Bubble sort by hand.

Hard

A shopping basket that filters, totals and reports.



	Stream	It teaches	Time
Python If-Else	A	if / elif / else on one number	10 min
Loops	A	for with range()	5 min
Counting Valleys	B	one pass with a counter	15 min

</>

Your turn

06

} /> [



Three minutes

What you built, one thing you are proud of, one thing that broke.

No slides needed. Show the code running.

Everyone else

One question per presentation, and it may not be a criticism.

Ask how, not why not.

Then applause

No grading theatre. Presenting your own work in front of peers is the skill being practised, not the code.





The diagram illustrates a code block structure. A large central rectangle contains the text `{ GAME }`. To its left is a yellow `</` symbol, and to its right is a white `/>` symbol. Below the central rectangle is a smaller rectangle containing the text `} /> [`. To the right of the central rectangle is another rectangle containing the text `Teams. Questions about code.`. Below this rectangle is a line of text `Highest score wins.`. The entire diagram is set against a dark background with white and yellow text and symbols.

`</` { GAME }

`/>`

`} /> [`

Teams. Questions about
code.

Highest score wins.



Everything you need was on a slide today.

Next week: functions, dictionaries,
and we leave Colab for VS Code.

</ Thanks!

Questions?
pejman.ebrahimi@uni.li



/>

} /> [